Conjugate Heat Transfer Simulations and Thermal Fault Detection in a Battery Pack using OpenFOAM

Undergraduate Research Project with Prof. Gandham Phanikumar

Ganesh B, ch22b002

Abstract

We perform conjugate heat transfer simulations of a liquid cooled battery pack using OpenFOAM. Starting with a much simpler case of a single body heat transfer problem, we incrementally simulate the complete system with all the battery cells, coolant jacket, and counter-current air flow. We then try to predict battery malfunction based on physically available temperature readings, by analyzing the thermal profile of the coolant. This is significant in real-world applications where temperature build up in a cell may result in catastrophe. In our system, we have a total of 4 cells, resulting in 15 different fault combinations. Our results show that cosine similarity of the coolant temperature profile vectors classifies these cases with perfect accuracy for all increasing temperature levels of the faulty cell(s) in non-noisy environments. In fact, we show that probing just two points out of the whole temperature profile is sufficient to achieve this classification.

Contents

1 Introduction **3**
- 1.1 Simulation Tools and Workflow . . . 3

2 Heat Transfer from a Single Heated Cylinder **3**
- 2.1 Objective . . . 3
- 2.2 Geometry and Mesh . . . 3
- 2.3 Results . . . 4
- 2.4 Observations . . . 5

3 Thermal Interaction in a Multi-Cylinder Array **5**
- 3.1 Objective . . . 5
- 3.2 Geometry and Mesh . . . 5
- 3.3 Results . . . 5

4 Multi-Region Coupling Benchmark **6**
- 4.1 Objective . . . 6
- 4.2 Geometry . . . 6
- 4.3 Results . . . 7
- 4.4 Observations . . . 8

5 Transient Simulation of the Battery Pack **8**
- 5.1 Objective . . . 8
- 5.2 Geometry and Mesh . . . 8
- 5.3 Results . . . 8

6 Steady-State Simulation **9**
6.1 Objective . . . 9
6.2 Results . . . 9

7 Introducing a Faulty Cell **10**
7.1 Objective . . . 10
7.2 Modeling Approach . . . 10
7.3 Results . . . 11

8 Parametric Study of all Fault Combinations **11**
8.1 Setup . . . 11
8.2 Temperature Profiles . . . 11

9 Fault Identification from Coolant Temperature Profiles **12**
9.1 Extracting Coolant Temperature Profiles . . . 12
9.2 Differential Profiles and Fault Signatures . . . 13
9.3 The Classification Algorithm . . . 14
9.4 Power Invariance through Normalization . . . 14
9.5 Separability of Fault Masks . . . 14
9.6 Validation . . . 15

10 Optimal Sensor Placement **16**
10.1 On the Number of Probes . . . 16
10.2 Unconstrained Best Pair . . . 16
10.3 Effect of Minimum Separation . . . 16

11 Future Work **18**

12 Conclusion **18**

1 Introduction

Lithium-ion battery packs are widely used in electric vehicles and energy storage systems, but they are vulnerable to thermal runaway when individual cells develop internal faults that cause excessive local heat generation. Detecting these faults early is essential for the battery management system to take appropriate actions before a dangerous cascade occurs.

In typical battery pack designs, the cells are sealed inside a housing and cannot be instrumented individually with temperature sensors. However, a liquid coolant flowing through the pack absorbs heat from every cell it passes, and the resulting temperature profile along the coolant channel contains information about the thermal state of each cell. We try to exploit this to identify the faulty cell(s).

We begin with very simple conjugate heat transfer systems, a single fluid flowing around a cylinder. We then build a grid of cells, test OpenFOAM's `chtMultiRegionFoam` tutorial case, and then move on to transient and steady-state simulations of the actual battery pack. Finally, we develop a classification algorithm based on 3 different power levels of the faulty cells $(2, 4, 6$ W), compared to the regular functioning cells at 1 W. The algorithm, as we will see, is independent of the faulty cell temperature, and achieves 100% accuracy on unseen cases.

1.1 Simulation Tools and Workflow

All simulations in this project were performed using OpenFOAM. The geometries were created and meshed in SALOME using `NETGEN-1D-2D-3D`, exported in UNV format, and made OpenFOAM compatible using `ideasUnvToFoam`. Steady-state cases used the `chtMultiRegionSimpleFoam` solver, while transient cases used `chtMultiRegionFoam`.

We proceed in the following way:

Stage 1. Heat transfer from a single heated cylinder (solver validation)

Stage 2. Heat transfer from a grid of nine cylinders (multi-heated body validation)

Stage 3. Multi-region heater benchmark (multi-fluid region validation)

Stage 4. Transient simulation of the battery pack geometry

Stage 5. Steady-state baseline of the healthy battery pack

Stage 6. Introduction of a single faulty cell

Stage 7. Parametric study of all fault combinations

Stage 8. Fault identification from coolant temperature profiles

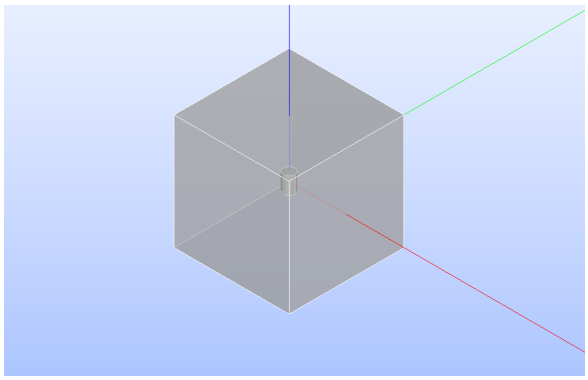
2 Heat Transfer from a Single Heated Cylinder

2.1 Objective

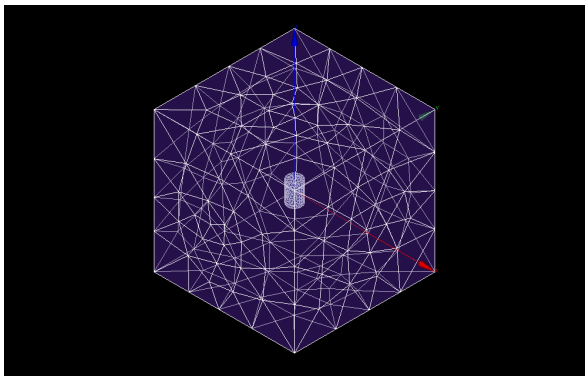
The first step is to set up and validate a basic conjugate heat transfer simulation. A single heat-generating solid cylinder is placed in a forced-convection fluid domain, and the simulation is used to verify that the solver correctly computes the coupled solid-fluid temperature field.

2.2 Geometry and Mesh

The domain consists of a cubic fluid region ($20 \times 20 \times 20$ mm) with a single cylindrical solid ($r = 1$mm, $h = 3$mm) at its center (Figure 1). The mesh is generated in SALOME with local refinement near the solid-fluid interface to resolve the thermal boundary layer.



(a) Geometry: inlet, outlet and 4 outer walls

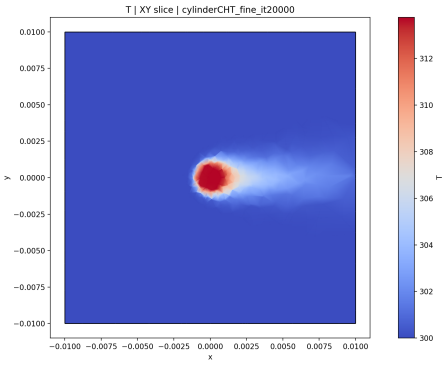


(b) Mesh with refinement at the solid-fluid interface

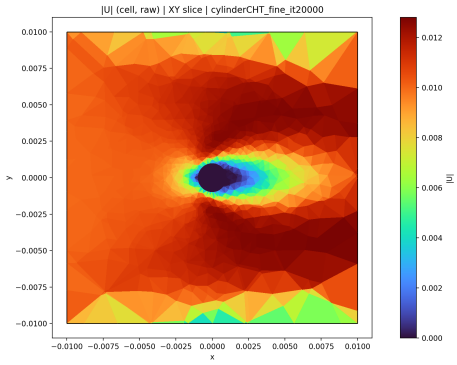
Figure 1: Single cylinder CHT: geometry and mesh

2.3 Results

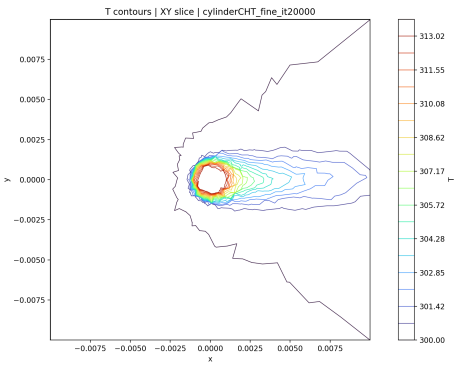
The simulation was run for 20000 iterations until the residuals dropped well below 10^{-4}. Figure 2 shows the temperature field and velocity magnitude on the mid-plane. The cylinder reaches approximately 313 K due to internal heat generation, while the bulk fluid remains near the inlet temperature of 300 K. A thermal wake is clearly visible downstream of the cylinder, where the heated fluid is carried along by the flow.



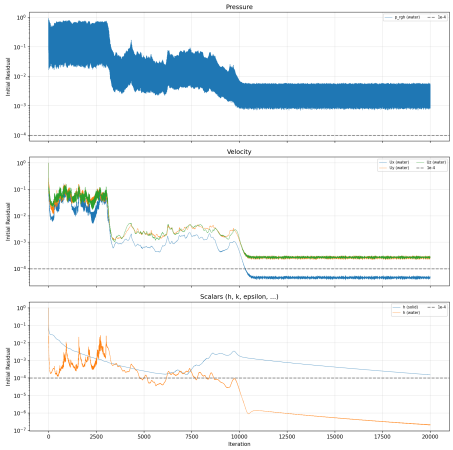
(a) Temperature field on the mid-plane



(b) Velocity magnitude



(c) Temperature contours



(d) Residual convergence

Figure 2: Single cylinder CHT results after 20000 iterations

2.4 Observations

- Mesh refinement at the solid–fluid interface is important for capturing the temperature gradient accurately.
- The thermal wake extends several diameters downstream, which suggests that in a multi-cell arrangement, downstream cells will be influenced by the heat from upstream ones.

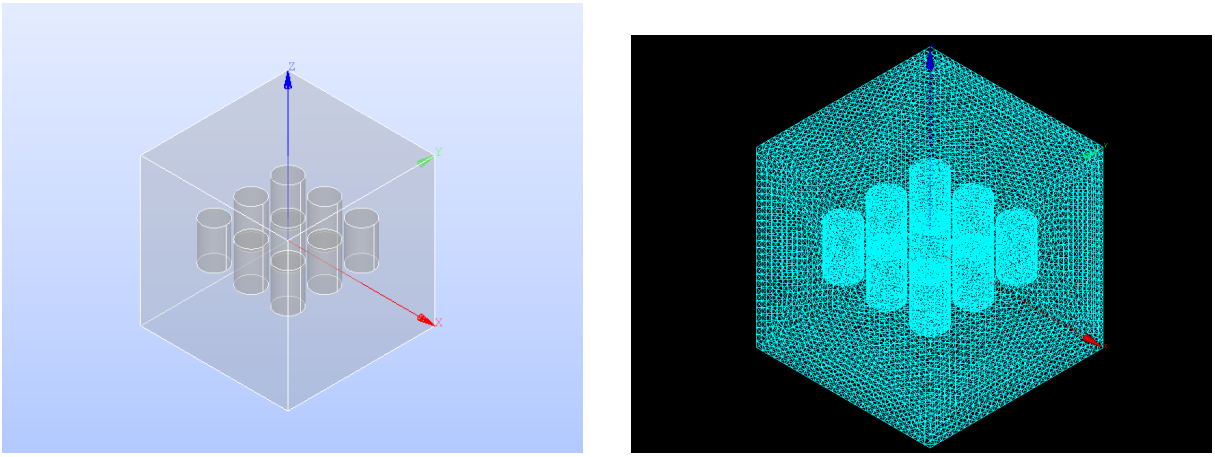
3 Thermal Interaction in a Multi-Cylinder Array

3.1 Objective

Having validated the solver on a single body, the next step is to study what happens when multiple heat-generating solids are placed close together. A 3×3 array of nine cylinders is simulated to investigate how neighboring cells influence each other's temperature.

3.2 Geometry and Mesh

Nine identical cylinders were arranged in a regular grid within a cubic fluid domain (Figure 3). Each cylinder generates the same amount of heat.

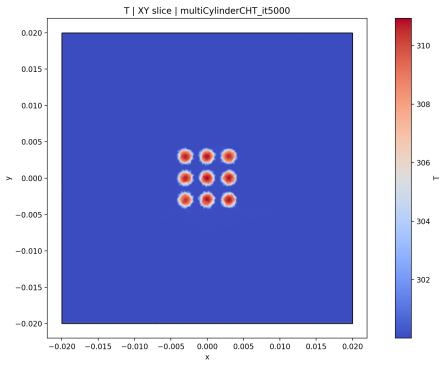


(a) Geometry: 3×3 cylinder array (b) Mesh

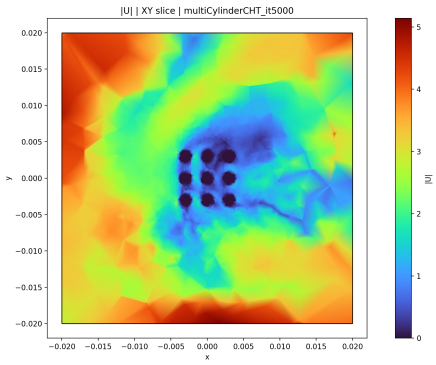
Figure 3: Multi-cylinder array: geometry and mesh

3.3 Results

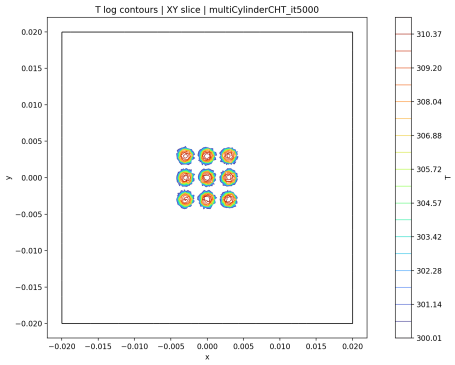
Figure 4 shows the converged solution after 30000 iterations. The most important observation is that the center cylinder reaches the highest temperature (approximately 320 K), while the corner cylinders stay cooler (approximately 315 K). This happens because the center cylinder is shielded from the incoming flow by its neighbors and has less convective access.



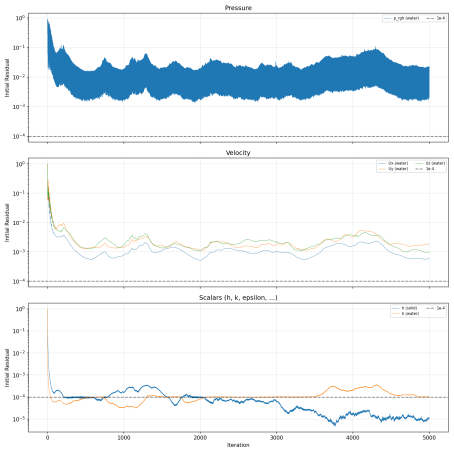
(a) Temperature field



(b) Velocity magnitude



(c) Temperature contours



(d) Residual convergence

Figure 4: Multi-cylinder CHT results after 5000 iterations

4 Multi-Region Coupling Benchmark

4.1 Objective

Before moving to the full battery pack geometry, we test multi-region coupling using OpenFOAM's tutorial case `multiRegionHeater`. This case has five distinct regions: two fluids (air and water) and three solids (a heater and two side blocks).

4.2 Geometry

The tutorial geometry (Figure 5) consists of a central heater block sandwiched between two solid side blocks, with air flowing above and water flowing below.

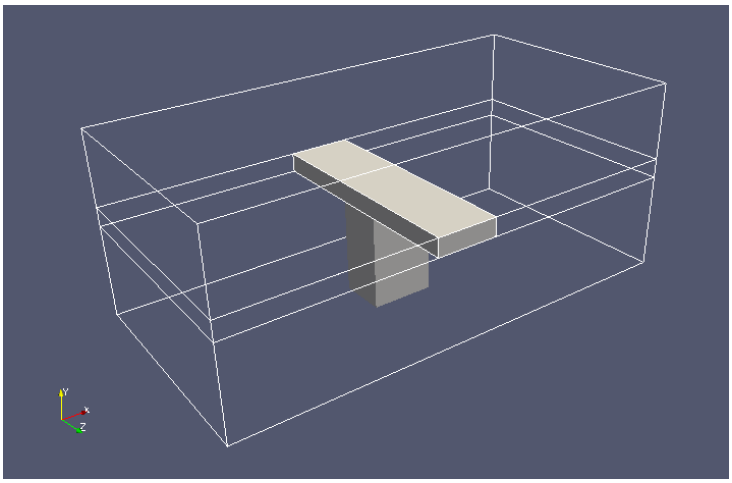
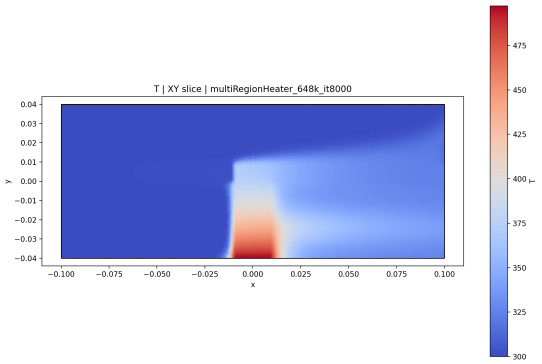


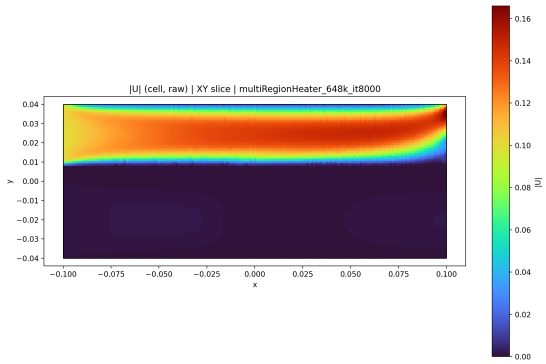
Figure 5: Multi-region heater geometry: using blockMesh

4.3 Results

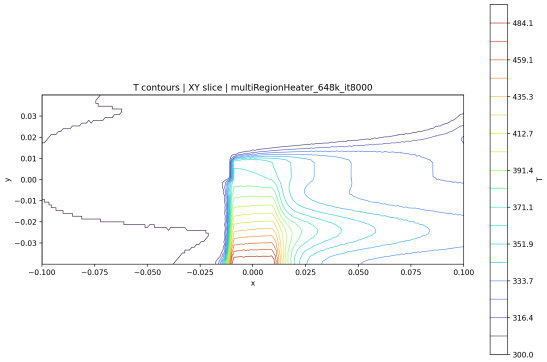
Figure 6 shows the converged temperature and velocity fields at 8000 iterations. The heater reaches approximately 475 K, with heat flowing through the solid blocks into both fluid streams. The water stream removes significantly more heat than the air stream, as expected from its much higher thermal conductivity and heat capacity.



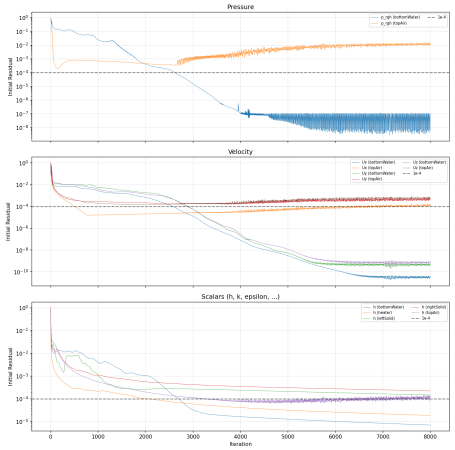
(a) Temperature field



(b) Velocity magnitude



(c) Temperature contours



(d) Residual convergence

Figure 6: Multi-region heater results

4.4 Observations

- This validates the simulation framework for the battery pack geometry, which similarly involves multiple solid cells, a liquid coolant channel, and an air domain.
- The strong asymmetry between air and water cooling confirms that the coolant channel is the primary heat removal path, which is why the coolant temperature profile is expected to carry the most useful information about cell health.

5 Transient Simulation of the Battery Pack

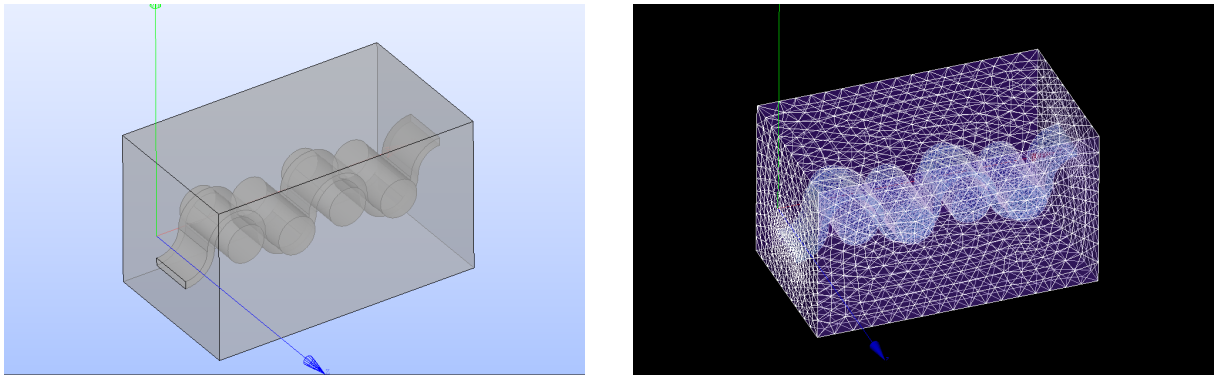
5.1 Objective

We now simulate our actual battery pack system. The geometry consists of four cylindrical cells in a row, cooled by a serpentine coolant channel that weaves between them, all enclosed in an air domain. We first use the transient solver to observe how the temperature field evolves from a uniform initial condition to thermal equilibrium. Note that we keep the air region at 0 velocity to improve numerical stability.

5.2 Geometry and Mesh

The geometry (Figure 7) contains three regions:

- **Air domain**: a rectangular enclosure surrounding the cells and coolant channel.
- **Coolant channel**: a serpentine passage weaving between the four cylindrical cells, carrying liquid coolant.
- **Battery cells**: four cylindrical solids.



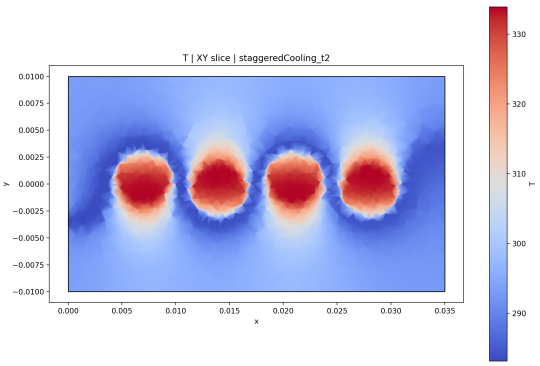
(a) 3D view of the battery pack geometry

(b) Computational mesh

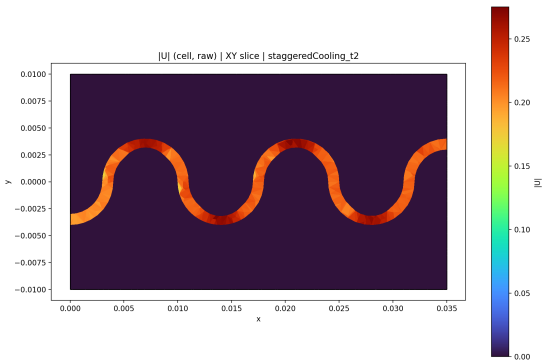
Figure 7: Battery pack geometry with serpentine coolant channel and four cells

5.3 Results

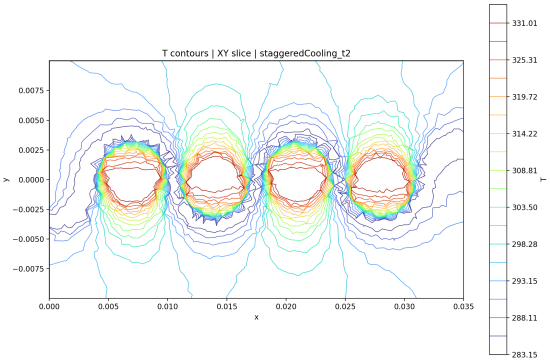
The transient simulation captures the thermal warm-up from a uniform initial temperature to a quasi-steady state (Figure 8). The cells heat up progressively as they generate heat, while the serpentine coolant channel removes heat. An important feature is the streamwise non-uniformity: upstream cells are cooled by colder coolant, while downstream cells see coolant that has already been warmed by the upstream cells. Cell temperatures range from approximately 300 K (upstream) to 330 K (downstream) under normal operation.



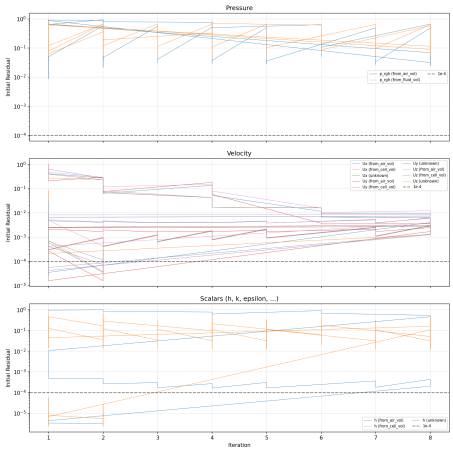
(a) Temperature field at the final time step



(b) Velocity magnitude in the coolant channel



(c) Temperature contours



(d) Residual convergence

Figure 8: Transient simulation of the battery pack

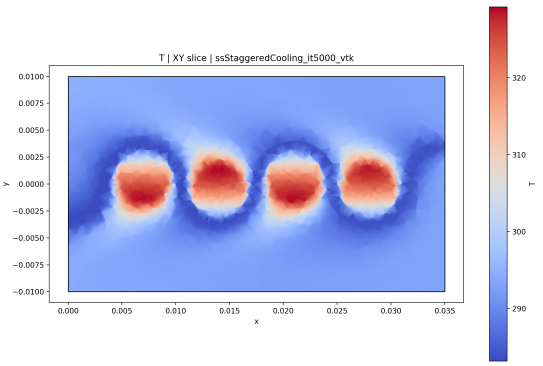
6 Steady-State Simulation

6.1 Objective

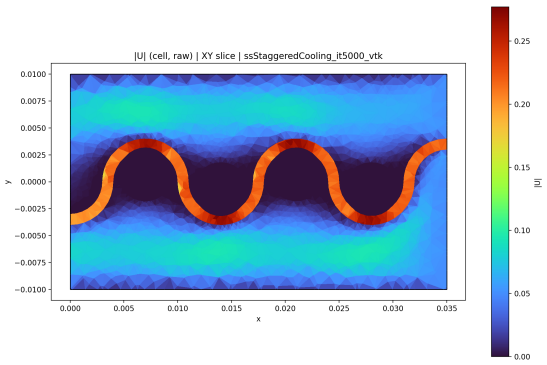
The transient solver from the previous stage is now replaced with a steady-state solver using `chtMultiRegionSimpleFoam`. This produces the **baseline healthy case** - the reference temperature field when all cells are operating normally, at a fraction of the computational cost.

6.2 Results

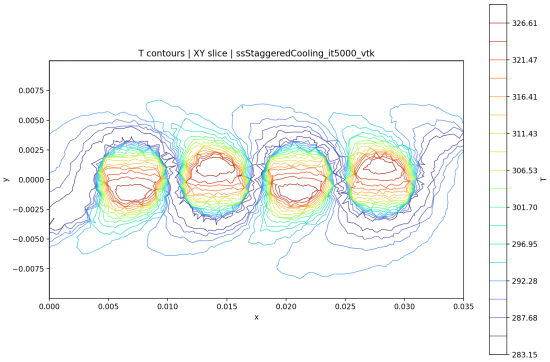
Figure 9 shows the converged steady-state solution at 5000 iterations. The temperature distribution is qualitatively the same as the transient result. Notice that we introduce a counter-current air flow in this case.



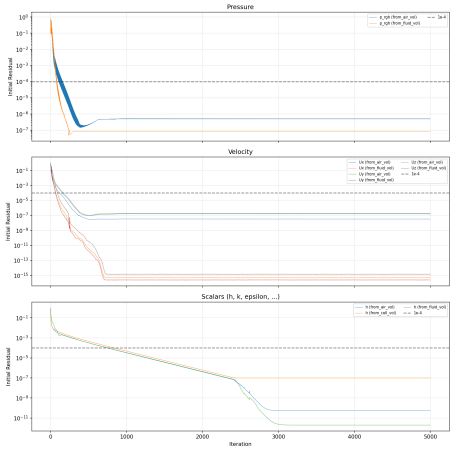
(a) Temperature field



(b) Velocity magnitude



(c) Temperature contours



(d) Residual convergence

Figure 9: Steady-state baseline (all cells healthy) at 5000 iterations. This solution serves as the reference for all fault detection calculations

The coolant enters the pack at 283.15 K and exits at 283.96 K, giving a total temperature rise of $\Delta T_{\text{coolant}} = 0.81\,\text{K}$ across the channel. This baseline profile $T_{\text{baseline}}(x)$ is subtracted from all faulty-case profiles in the subsequent analysis. While this is a relatively small increase in temperature due to the fast coolant flow, this is sufficient to study the general properties of this system.

7 Introducing a Faulty Cell

7.1 Objective

With the healthy baseline established, the next step was to simulate what happens when one cell develops an internal fault. A fault is modeled as an increase in the cell's volumetric heat generation rate, representing the excess heat produced by an internal short circuit or other degradation mechanism.

7.2 Modeling Approach

The fault was introduced by modifying `fvOptions`, which allows a volumetric source term to be added to a specific cell region. The excess heat generation was set to 2 W, 4 W, or 6 W above the nominal rate. All other cells continued to operate at their normal power of 1 W.

7.3 Results

Figure 10 shows the temperature field when cell 1 is faulty (mask `1000`) with 6 W of excess power. The faulty cell reaches approximately 500 K: far above the 320 K of the healthy cells, and produces a pronounced thermal plume in the coolant downstream. This confirms that a cell fault produces a detectable thermal signature in the coolant.

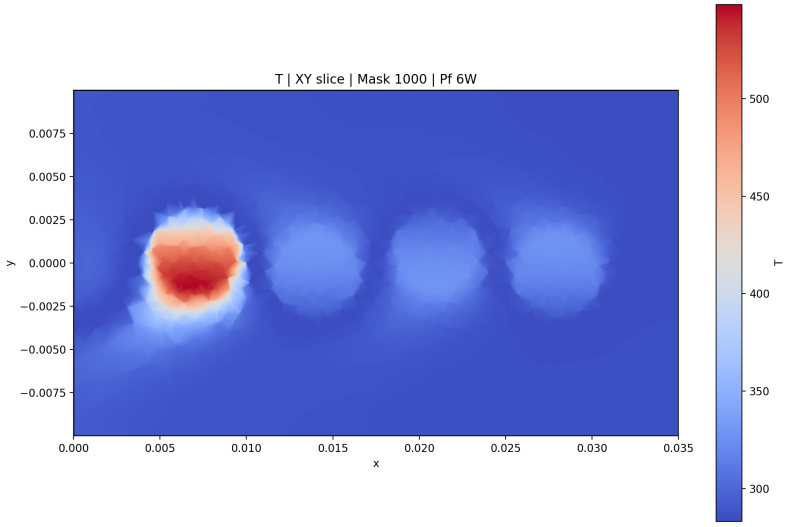


Figure 10: Temperature field with cell 1 faulty (mask `1000`, 6 W excess power). The faulty cell reaches approximately 500 K and creates a thermal plume in the downstream coolant

8 Parametric Study of all Fault Combinations

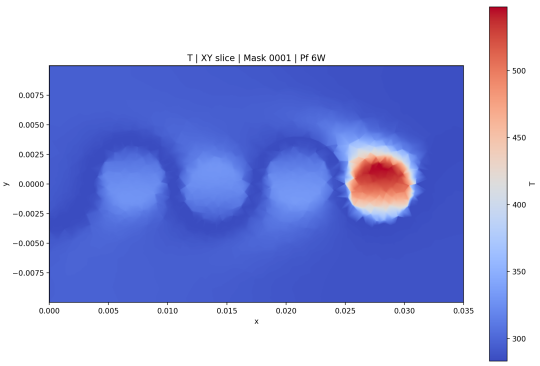
8.1 Setup

With four cells, each either healthy (0) or faulty (1), there are $2^4 - 1 = 15$ non-trivial fault combinations (excluding the all-healthy baseline). Each combination is described by a 4-bit binary mask, where a `1` in position i means cell i is faulty. For example, mask `0110` means cells 2 and 3 are faulty.
Each of the 15 masks was simulated at four power levels:

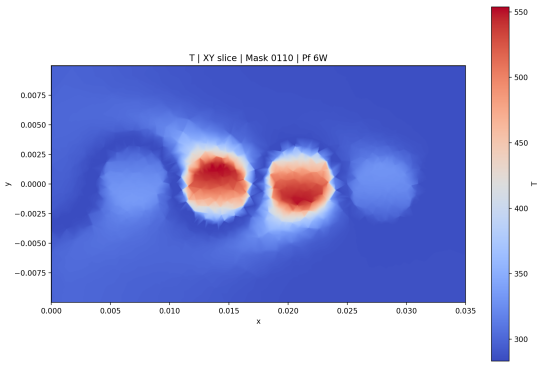
- **Training set**: 2 W, 4 W, and 6 W ($15 \times 3 = 45$ simulations).
- **Test set**: 8 W ($15 \times 1 = 15$ simulations), held out for blind validation.
- **Baseline**: 1 simulation with no faults.
- **Total**: 61 steady-state CHT simulations.

8.2 Temperature Profiles

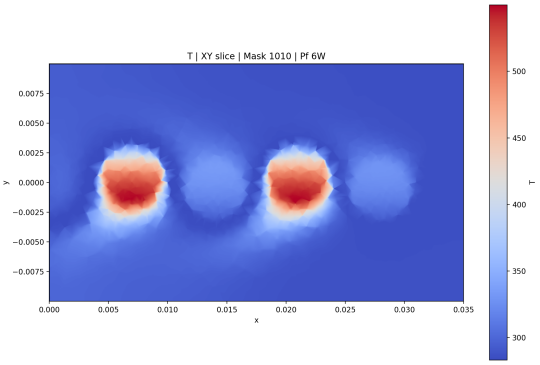
Figure 11 shows the temperature fields for four representative fault masks at 6 W. Each combination produces a visually distinct thermal pattern, with hot spots appearing at the locations of the faulty cells and thermal plumes extending downstream through the coolant.



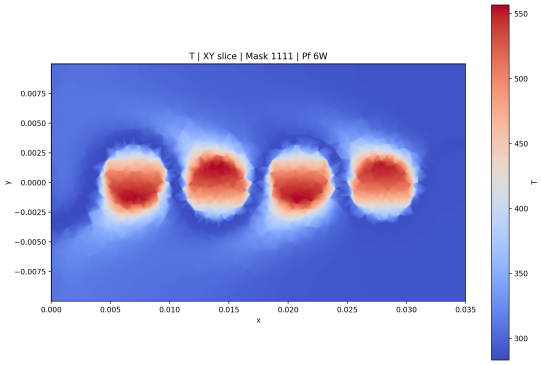
(a) Mask `0001`: cell 4 faulty



(b) Mask `0110`: cells 2 and 3 faulty



(c) Mask `1010`: cells 1 and 3 faulty



(d) Mask `1111`: all four cells faulty

Figure 11: Temperature profiles for selected fault masks at 6 W. Each fault combination produces a distinct thermal pattern

9 Fault Identification from Coolant Temperature Profiles

The goal is to determine which cell(s) are faulty using only temperature measurements from the coolant channel. Direct measurement of individual cell temperatures is assumed to be unavailable, as is typical in sealed battery pack designs. The input to the classification algorithm is the coolant bulk temperature profile $T(x)$ measured at discrete points along the serpentine channel.

9.1 Extracting Coolant Temperature Profiles

The coolant bulk temperature was extracted along the streamwise direction by dividing the channel into 120 uniformly spaced bins.

Figure 12 shows the resulting profiles for all 46 training cases (15 masks at 3 power levels, plus the baseline), grouped by power level.

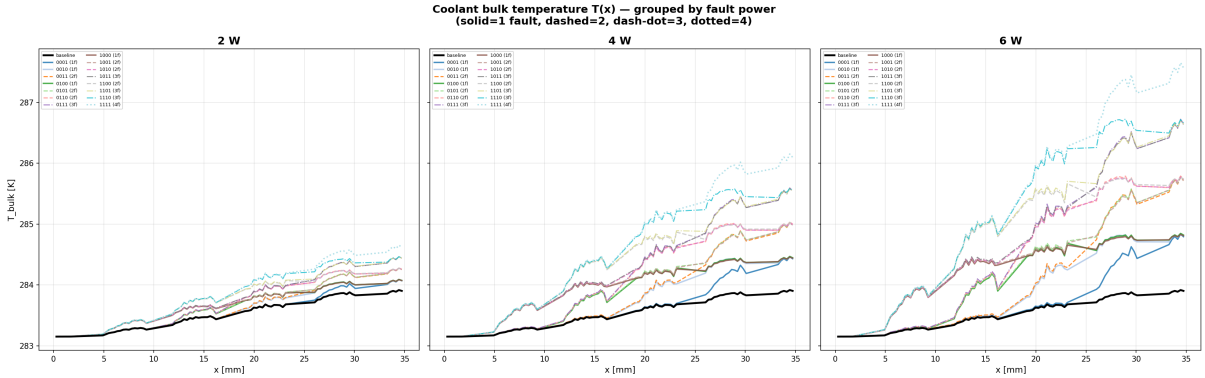


Figure 12: Coolant bulk temperature profiles for all 46 training cases, grouped by power level. Line style indicates the number of faulty cells: solid = 1, dashed = 2, dash-dot = 3, dotted = 4

9.2 Differential Profiles and Fault Signatures

To isolate the effect of the fault, the baseline profile is subtracted from each faulty-case profile:

$$\Delta T(x) = T_{\text{faulty}}(x) - T_{\text{baseline}}(x)$$

Figure 13 shows the resulting differential profiles. Each faulty cell produces a step-like increase in ΔT at its streamwise position, because the extra heat is absorbed by the coolant and carried downstream. When multiple cells are faulty, the steps accumulate, producing a staircase-like pattern.

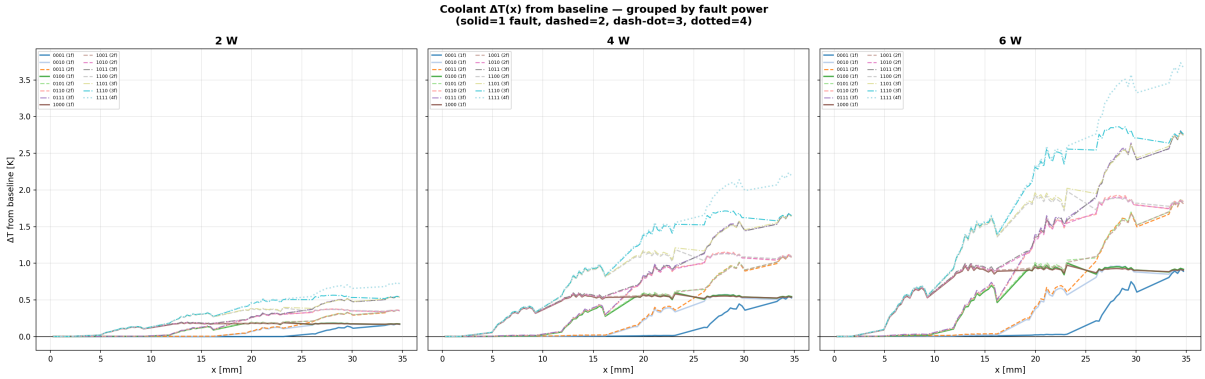


Figure 13: Differential coolant profiles $\Delta T(x)$. Each fault mask produces a unique staircase pattern. The step heights scale with power, but the step locations depend only on which cells are faulty

Three important observations emerge from this plot:

- Each faulty cell contributes a step of approximately 0.15 K (at 2 W) to 0.9 K (at 6 W). Note that we only had an overall increase of 0.81 K in the normal case.
- The shape of the $\Delta T(x)$ profile, that is, where the steps occur and their relative heights, depends on which cells are faulty.
- The amplitude of the profile scales with the fault power, but the shape remains the same across different power levels.

9.3 The Classification Algorithm

Based on the power-invariance observation, we propose the following algorithm:

1. **Subtract the baseline**: compute $\Delta T(x) = T_{\text{new}}(x) - T_{\text{baseline}}(x)$ to remove flow dependence.
2. **Normalize**: divide by the L2 norm to remove the power dependence, giving $\hat{T} = \Delta T / \|\Delta T\|_2$.
3. **Compare**: compute the cosine similarity between $\hat{T}$ and every normalized training profile.
4. **Classify**: assign the fault mask of the most similar training profile (1-nearest-neighbor).

The cosine similarity between two vectors $\mathbf{a}$ and $\mathbf{b}$ measures how closely their directions align, regardless of their magnitudes:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

This metric was chosen because it naturally identifies which directions (components) change the most.

9.4 Power Invariance through Normalization

Figure 14 demonstrates the effect of normalization. In the left panel, the raw ΔT profiles for the same mask at different power levels are spread apart because the amplitude scales with power. After L2 normalization (center panel) or peak normalization (right panel), the profiles for the same mask collapse onto a single curve, while profiles for different masks remain well separated.

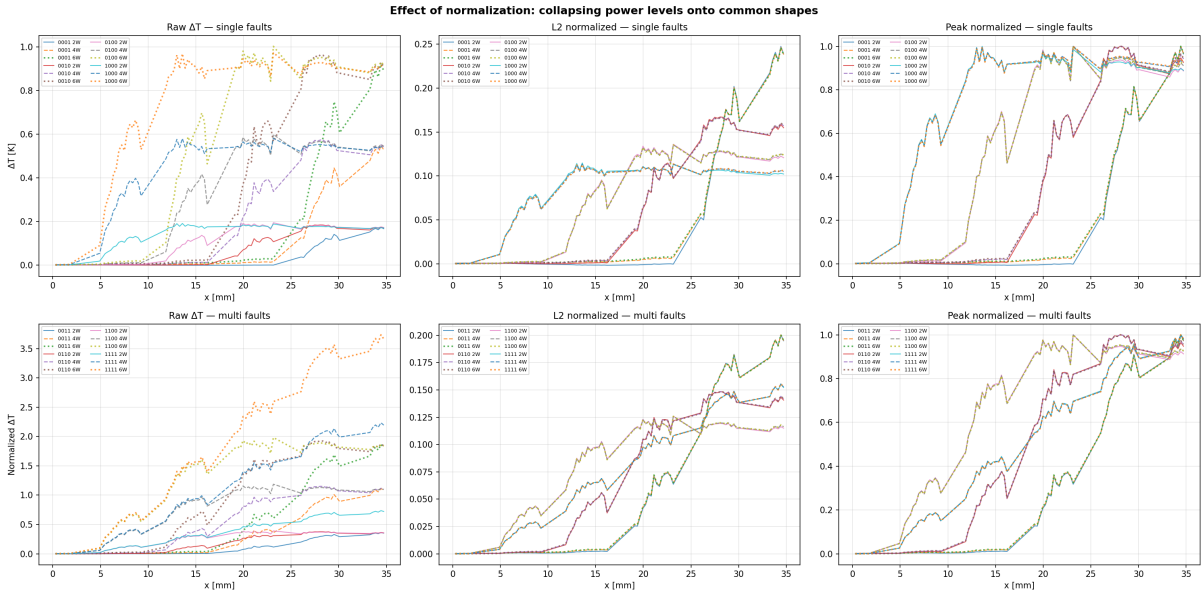


Figure 14: The effect of normalization on ΔT profiles. Left: raw profiles are power-dependent. Center: L2 normalization collapses same-mask profiles. Right: peak normalization achieves a similar result

9.5 Separability of Fault Masks

To confirm that all 15 fault masks are distinguishable from each other, Figure 15 shows the pairwise cosine similarity between the normalized mean profile of each mask. The diagonal entries

are 1.0 (each mask is identical to itself), and the maximum off-diagonal entry is approximately 0.98, meaning no two masks have identical signatures.

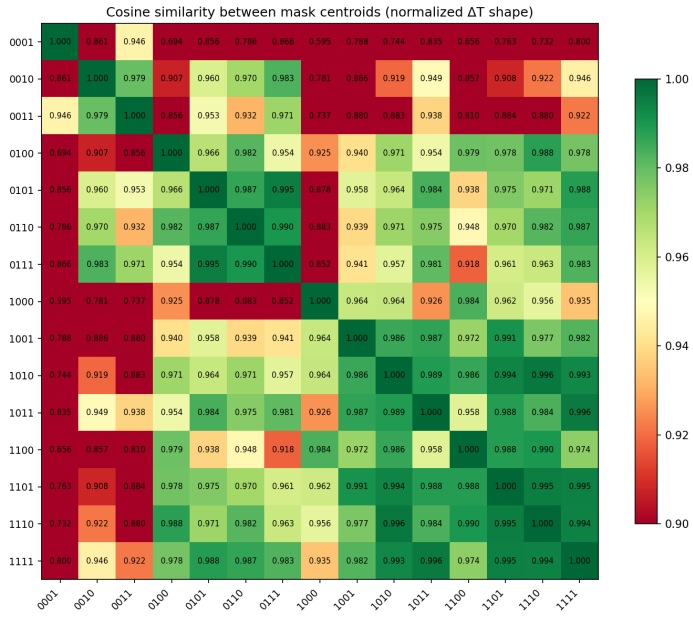


Figure 15: Pairwise cosine similarity between the 15 normalized mean profiles. All masks are distinguishable, with the maximum off-diagonal similarity at approximately 0.98

9.6 Validation

The classifier was trained using the 45-case library (15 masks at 2, 4, and 6 W) and then tested on 15 new simulations at 8 W, a power level that was not used for training. If the normalized ΔT shape depends only on which cells are faulty and not on how much power the fault generates, then the classifier should generalize to the unseen power level.

Every single test case was classified correctly, with cosine similarities exceeding 0.9999. This confirms that the normalized ΔT shape is independent of the fault power level, and that a training library built at lower power levels generalizes perfectly to elevated, unseen power levels.

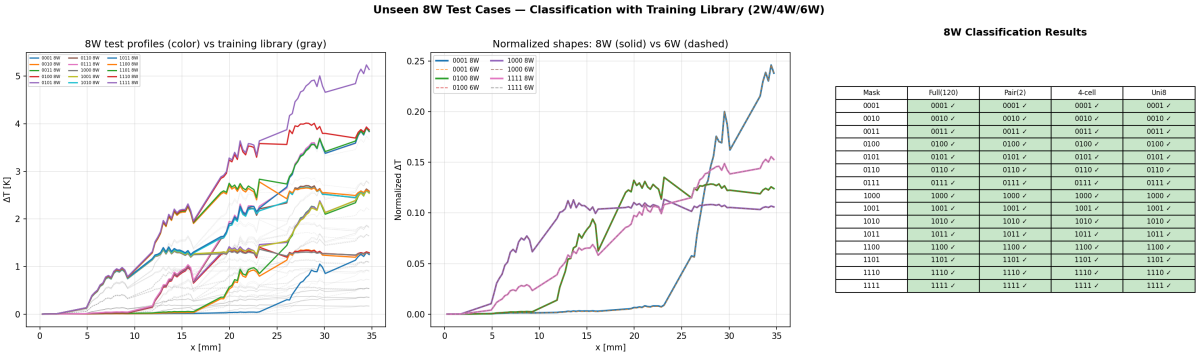


Figure 16: Blind test at 8 W. Left: test profiles (colored) overlaid on the training library (gray). Center: normalized profiles at 8 W (solid) vs. 6 W (dashed) confirm shape invariance. Right: per-mask classification results

10 Optimal Sensor Placement

A practical deployment of this method requires physical temperature sensors to be installed in the coolant channel. This section investigates how many sensors are needed and where they should be placed.

10.1 On the Number of Probes

An exhaustive search was performed over all possible combinations of probe locations drawn from the 114 non-trivial bins (excluding the inlet region where $\Delta T \approx 0$).

- **1 probe**: the best single probe achieves only 6.7% accuracy, which is as poor as random guessing.
- **2 probes**: 222 probe pairs achieve 100% accuracy. Two measurements are sufficient because they capture both the position and relative magnitude of the steps.
- **3 probes**: thousands of triplets achieve 100% accuracy.

10.2 Unconstrained Best Pair

Without any constraint on probe spacing, the exhaustive search finds the best pair at **(29.5 mm, 30.4 mm)**, two points less than 1 mm apart near the outlet of the coolant channel. This pair achieves 100% accuracy because the ΔT values at these two positions happen to separate all 15 fault masks in the resulting 2D feature space.

However, placing two sensors less than 1 mm apart is impractical: finite sensor size, thermal crosstalk, and measurement noise would degrade the separation that the exhaustive search assumes. A minimum separation constraint is therefore imposed.

10.3 Effect of Minimum Separation

Table 1 shows the number of perfect probe pairs as a function of the minimum allowed separation. Even with a 15 mm constraint, 30 pairs still achieve 100% accuracy.

Table 1: Number of probe pairs achieving 100% accuracy as a function of minimum separation

Min. separation	Perfect pairs	Best pair
0 mm (unconstrained)	222	(29.5, 30.4 mm); 0.9 mm apart
3 mm	135	(4.3, 24.9 mm); 20.5 mm apart
5 mm	109	(4.3, 24.9 mm); 20.5 mm apart
10 mm	78	(4.3, 24.9 mm); 20.5 mm apart
15 mm	30	(4.3, 24.9 mm); 20.5 mm apart

Once any spatial constraint is imposed, the pair **(4.3 mm, 24.9 mm)** naturally emerges as the best choice. This places one probe just downstream of cell 1 and the other just downstream of cell 3, separated by 20.5 mm (Figure 17).

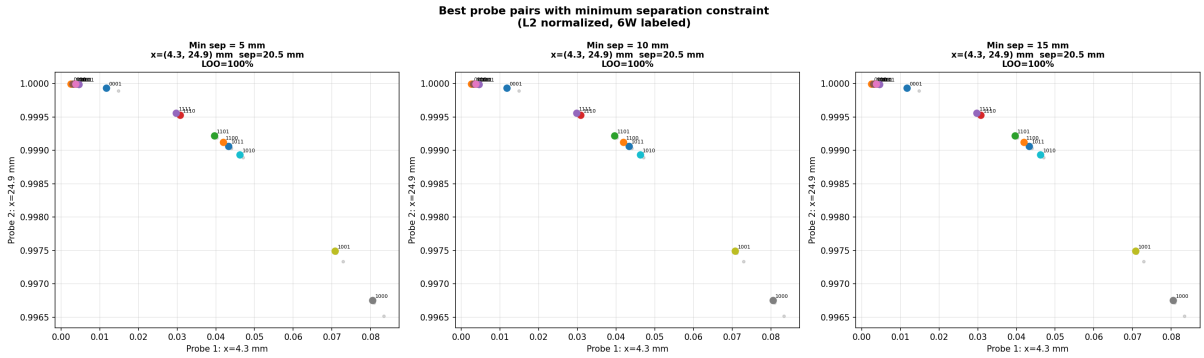


Figure 17: Feature space formed by the best probe pair under a minimum separation constraint. All 15 fault masks are cleanly separated

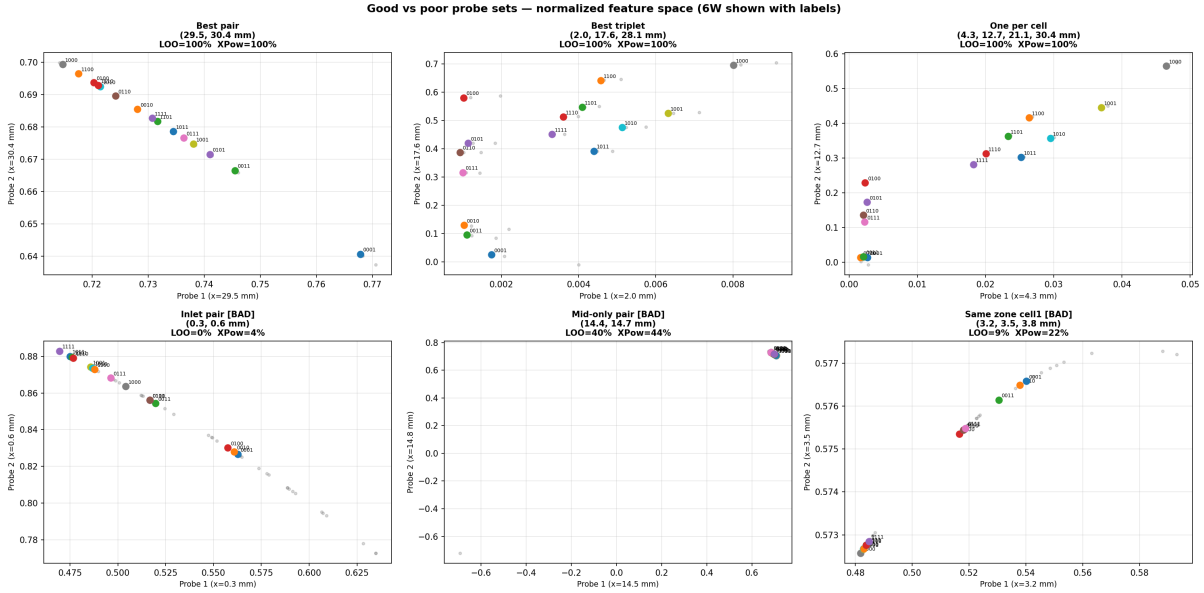


Figure 18: Feature space for various probe configurations

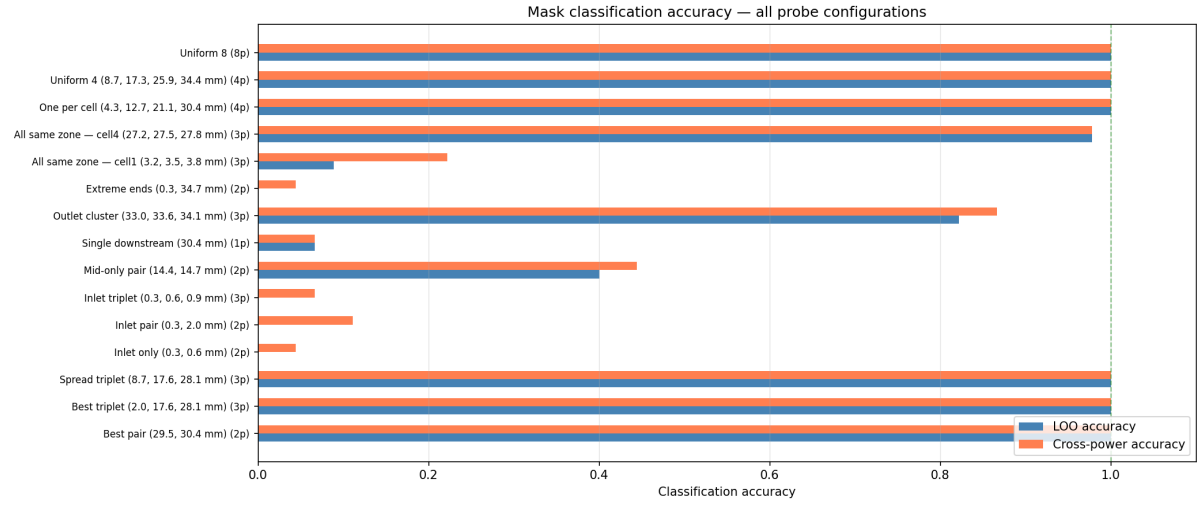


Figure 19: Summary of classification accuracy across all tested probe configurations

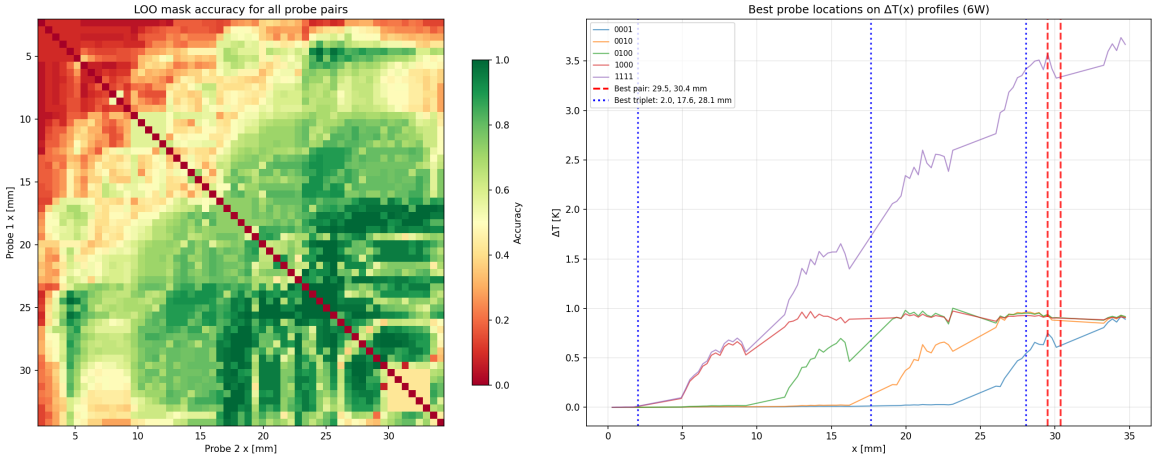


Figure 20: Probe pair accuracy heatmap (left) showing which combinations achieve 100% accuracy, and the optimal probe locations marked on the ΔT profiles (right)

11 Future Work

The present study considers a fixed four-cell geometry, prescribed operating conditions, and noiseless simulated measurements, with equal excess power assigned to all faulty cells in each case. Future work could extend the analysis to arbitrary grid arrangements and different coolant-channel geometries. This would establish how broadly the methodology applies and how sensor requirements change with the configuration.

Further studies should consider different coolant inlet and ambient temperatures, flow rates, and independently varying fault powers. Cosine similarity is effective here because the normalized profiles remain approximately unchanged under uniform scaling of fault power. Whether this property persists under more general conditions is unknown. If the signatures change substantially, alternative methods, like machine-learning models trained across varied conditions, could be investigated.

Finally, sensor noise, calibration offsets, and placement uncertainty should be incorporated to assess robustness, particularly for weak faults. Sensor locations could then be selected to maximize separation under uncertainty. These tests, followed by transient studies and experimental validation, would help determine whether developing faults can be detected sufficiently early.

12 Conclusion

In this project, we developed a complete framework for detecting faulty cells in a liquid-cooled battery pack using only coolant temperature measurements. We built it incrementally: from a single heated cylinder to a full four-cell battery pack with a serpentine coolant channel and counter-current air flow. Each stage validated a specific aspect of the conjugate heat transfer setup before adding complexity.

Under the tested conditions, each fault combination produced a distinct differential coolant temperature profile. These profiles remained approximately invariant after normalization when the excess power of all faulty cells was scaled uniformly. A nearest-neighbor classifier using cosine similarity, trained on cases at 2, 4, and 6 W excess power, correctly identified all 15 fault combinations at the held-out level of 8 W.

An exhaustive search over probe locations also identified pairs that achieved perfect classification of the noiseless simulated cases. With a minimum spacing constraint, the selected pair

was located at 4.3 mm and 24.9 mm. This suggests that sparse coolant temperature measurements can retain useful information about fault locations. However, the practical sufficiency of two sensors remains to be established under measurement uncertainty, unequal fault powers, and varying operating conditions. The results therefore provide a numerical proof of concept for coolant-based fault identification and a basis for further studies and experimental validation.